

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

PipeWatch

CI/CD Pipeline and AWS Infrastructure Monitoring Platform

1. Introduction

1.1 Purpose

This document specifies the software requirements for PipeWatch, a cloud-based monitoring platform that provides a unified, real-time view of Jenkins CI/CD pipeline execution and AWS infrastructure health. It defines the functional and non-functional requirements of the system as implemented, describes the system workflow, and documents the technology stack used to build the solution.

1.2 Project Overview

PipeWatch integrates a Jenkins CI/CD pipeline hosted on AWS EC2 with an event-driven AWS backend (EventBridge, Lambda, DynamoDB, API Gateway) and a Node.js/Express REST API layer. The processed data is displayed on a React.js dashboard that gives developers a single screen to monitor pipeline status, deployment history, and infrastructure health instead of switching between Jenkins, the AWS Console, and terminal logs.

1.3 Objectives

- Provide a centralized dashboard for monitoring CI/CD pipeline execution and AWS infrastructure.
- Give real-time visibility into build status, current pipeline stage, and deployment history.
- Capture pipeline events automatically using AWS EventBridge, Lambda, and DynamoDB.
- Reduce the time required to detect pipeline or deployment failures.
- Send automated notifications for key pipeline events using an n8n workflow.

1.4 Scope

The scope of PipeWatch covers the Jenkins pipeline configuration, integration with AWS services (IAM, EC2, S3, EventBridge, Lambda, DynamoDB, API Gateway), a Node.js/Express backend, and a React.js dashboard. The system focuses on monitoring and visibility for an existing CI/CD and cloud setup; it does not replace Jenkins or AWS, and it monitors a single Jenkins pipeline deploying a static website to Amazon S3.

2. System Workflow

The complete workflow, as implemented, is described below.

2.1 Source Control and Trigger

- A developer pushes code changes to the project's GitHub repository.
- A GitHub webhook, configured on the repository, automatically triggers the Jenkins pipeline on every push.

2.2 Jenkins CI/CD Pipeline (hosted on EC2)

- Checkout – Jenkins pulls the latest source code from the GitHub repository.
- Validate HTML – the website files are checked for structural correctness before deployment.
- Deploy to S3 – the validated build is uploaded to the S3 static hosting bucket using the AWS CLI (aws s3 sync).
- Publish Event – the pipeline sends a deployment event to the AWS EventBridge custom event bus.
- Post Actions – the workspace is cleaned up and the final build status is reported.

2.3 Event-Driven AWS Processing

- EventBridge receives the deployment event from Jenkins and routes it to the processing Lambda function based on the event rule.
- The Lambda function parses the event payload, extracts build and deployment details, and writes a structured record to DynamoDB.
- DynamoDB stores the deployment and pipeline event record, forming the deployment history.
- API Gateway exposes REST endpoints that read the stored records from DynamoDB for consumption by the backend and dashboard.

2.4 Backend Processing

- The Node.js/Express backend polls the Jenkins REST API for build number, build status, current stage, queue length, and executor status.
- The backend also reads processed event and infrastructure data through the API Gateway endpoints.
- Raw responses are normalized into a consistent JSON structure (for example, converting Jenkins build results into a simplified success, failure, or in-progress status) and briefly cached before being served through REST APIs.

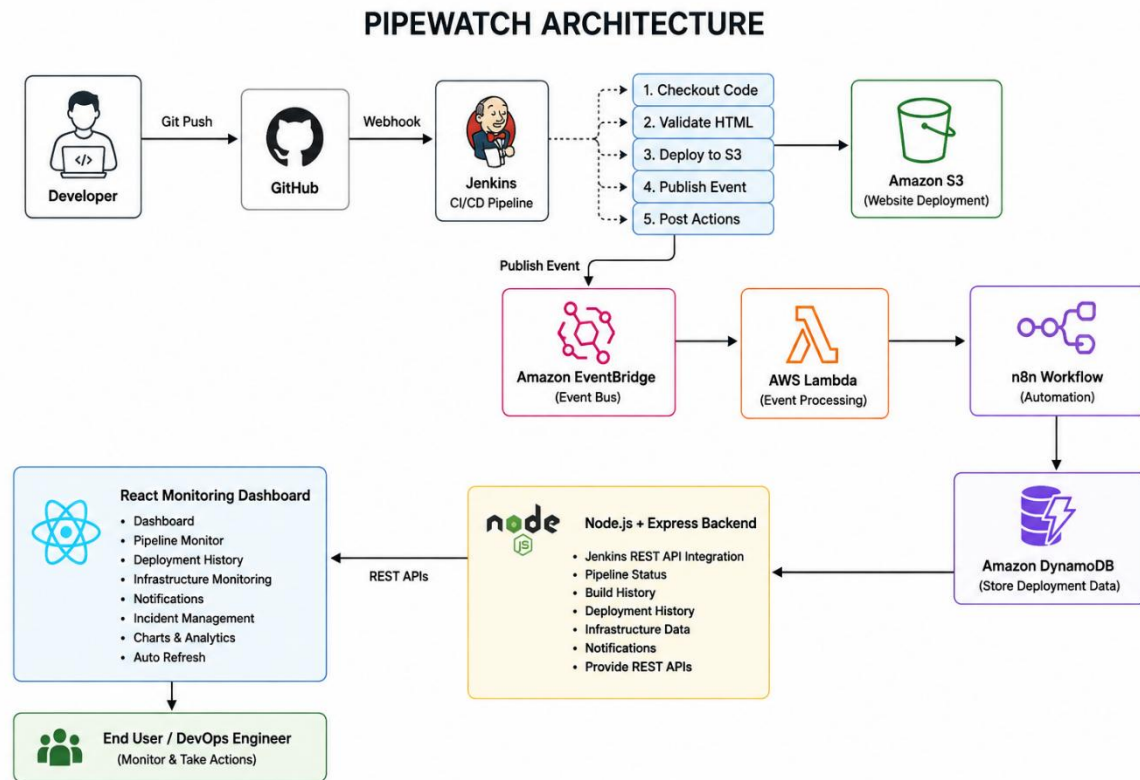
2.5 Dashboard Update

- The React.js dashboard requests data from the backend REST APIs on page load.
- An auto-refresh hook polls the backend APIs at fixed intervals without requiring a manual page reload.
- New data updates the KPI cards, tables, and charts in place, giving the user a live view of pipeline, deployment, and infrastructure status.

2.6 Notifications

- An n8n workflow listens for new deployment events, formats the event data into a readable message, and sends a notification when a deployment succeeds or fails.
- Recent pipeline and deployment events are also surfaced on the dashboard's Notifications page.

3. System Architecture



4. Functional Requirements

4.1 Pipeline Monitoring

- The system shall display the latest Jenkins build number, build status, and current pipeline stage.
- The system shall display build duration, queue length, and executor status, refreshed automatically.

4.2 Deployment History

- The system shall list past deployments with build number, commit details, deployment status, timestamp, and environment, retrieved from the DynamoDB-backed history API.

4.3 Infrastructure Monitoring

- The system shall display the live health status of EC2, S3, Lambda, API Gateway, EventBridge, and DynamoDB through the backend's infrastructure endpoint.

4.4 Event Processing

- The system shall capture deployment events published by Jenkins through EventBridge.
- The system shall process each event using a Lambda function and store a structured record in DynamoDB.

4.5 Data Access APIs

- The backend shall expose REST endpoints: GET /api/pipeline/status, GET /api/pipeline/history, GET /api/infrastructure, and GET /api/metrics.

- API access shall be protected using a token/key-based check, and AWS calls shall use restricted IAM credentials.

4.6 Dashboard Visualization

- The system shall render KPI cards for latest build number, status, success rate, and failure rate.
- The system shall render charts for pipeline trend, success rate, failure rate, build duration, deployment trend, and infrastructure usage using Recharts.
- The system shall auto-refresh dashboard data at fixed intervals without a manual page reload.

4.7 Incident Visibility

- Failed builds or deployments shall be flagged distinctly on the dashboard so they can be tracked separately from general history.

4.8 Notifications

- The system shall trigger an n8n workflow to send an automated notification when a deployment succeeds or fails.

5. Non-Functional Requirements

5.1 Performance

- The dashboard reflects a new Jenkins build status within a few seconds of pipeline completion.
- Backend API responses are served quickly using lightweight caching to support frequent auto-refresh polling.

5.2 Scalability

- Serverless components (Lambda, EventBridge, DynamoDB, API Gateway) scale automatically with event volume without manual server management.

5.3 Reliability

- Centralized error-handling middleware in the backend catches failures from Jenkins and AWS API calls (timeouts, authentication errors, unreachable services) and returns a consistent error response instead of failing silently.

5.4 Security

- A dedicated IAM user with least-privilege policies is used for all programmatic AWS access instead of root credentials.
- EC2 security group rules restrict inbound access to SSH (port 22) and Jenkins (port 8080) to required IP ranges.
- Backend API routes are protected with a token/key-based access check.

5.5 Availability

- The S3 static website hosting bucket serves the deployed application with high availability.
- Event processing through EventBridge and Lambda runs only when triggered, keeping the event pipeline available on demand.

5.6 Maintainability

- The backend follows a layered folder structure (routes, controllers, services, middleware, utils) separating concerns for easier maintenance.
- The frontend follows a component-based structure (components, pages, charts, services, hooks) for modular updates.

5.7 Usability

- Client-side routing separates the dashboard into distinct views (Dashboard, Pipeline Monitor, Deployment History, Infrastructure, Notifications) under a common layout and navigation bar.
- Colour-coded status badges and KPI cards allow build and deployment status to be understood at a glance.

6. Dashboard Features

6.1 Dashboard (Home)

Aggregates key information into KPI cards and summary charts for an at-a-glance view of overall system health.

6.2 Pipeline Monitor

Displays the latest build number, build status, current pipeline stage, build duration, queue length, and executor status, refreshed automatically.

6.3 Deployment History

Lists past deployments with build number, commit details, deployment status, timestamp, and environment, sourced from the DynamoDB-backed history API.

6.4 Infrastructure

Shows the live health of EC2, S3, Lambda, API Gateway, EventBridge, and DynamoDB, pulled through the backend's infrastructure endpoint.

6.5 Notifications

Surfaces recent pipeline and deployment events on the dashboard so the user does not need to check n8n or email separately.

6.6 Charts

- Pipeline Trend – build outcomes over time.
- Success Rate – percentage of successful builds.
- Failure Rate – percentage of failed builds.
- Build Duration – time taken per build, plotted over recent builds.
- Deployment Trend – frequency of deployments over time.
- Infrastructure Usage – health and activity of AWS services over time.

7. Technology Stack

7.1 Frontend

Component	Technology Used
Framework	React.js
Build Tool	Vite
Charts	Recharts
Styling	CSS

7.2 Backend

Component	Technology Used
Runtime	Node.js
Web Framework	Express.js

7.3 DevOps

Component	Technology Used
Source Control	GitHub (with webhook trigger)
CI/CD Server	Jenkins (Declarative Pipeline)
Workflow Automation	n8n

7.4 AWS Services

Component	Technology Used
IAM	Access control for all AWS resources
EC2	Hosts the Jenkins server
S3	Static website hosting and deployment target
EventBridge	Custom event bus routing deployment events to Lambda
Lambda	Processes deployment events and writes records to DynamoDB
API Gateway	Exposes REST endpoints for stored deployment data

7.5 Database

Component	Technology Used
DynamoDB	Stores deployment and pipeline event records with a partition key on build/event ID

7.6 Automation

Component	Technology Used
n8n Workflow	Listens for deployment events and sends automated notifications on success or failure

7.7 Monitoring

Component	Technology Used
Jenkins REST API	Polled by the backend for build number, status, stage, queue length, and executor status
React Dashboard	Auto-refreshing KPI cards, tables, and charts for pipeline and infrastructure monitoring

8. Expected Outcome

PipeWatch was implemented as a fully functional monitoring platform that displays live Jenkins pipeline status, deployment history, and AWS infrastructure health on a single React dashboard, backed by an automated event pipeline using EventBridge, Lambda, and DynamoDB, with notifications delivered through an n8n workflow.

The completed system removes the need to switch between the Jenkins console, the AWS Console, and terminal logs during a deployment. A new Jenkins build status is reflected on the dashboard within a few seconds of pipeline completion, deployment history is available with commit and environment details, and AWS infrastructure health is visible in real time. The event-driven backend keeps processing cost-efficient, since Lambda functions run only when a deployment event occurs, and the auto-refreshing dashboard gives the team consolidated analytics on build duration, success rate, and deployment frequency without manual monitoring.